



UNSW
SYDNEY

COMP3153/9153

Algorithmic Verification

Lecture 3: Expressiveness, CTL Model Checking

Administrivia

- Formatif tasks
- Feedback

Automata

- Deterministic and non-deterministic finite automata
- Product, synchronization via message passing
- Kripke structures

LTL

Linear Temporal Logic – specifies linear time properties.

LTL Syntax:

- Propositional logic plus **X** and **U**:

$$\varphi := p \parallel \varphi \wedge \varphi \parallel \neg \varphi \parallel \mathbf{X}\varphi \parallel \varphi \mathbf{U} \varphi$$

- Useful derived operations $\vee$, $\rightarrow$, $\leftrightarrow$, true, false, **F**, **G**

LTL Semantics:

- Trace: infinite sequence, σ , of sets of propositional variables
- $\sigma \models \varphi$ defined recursively (on structure of φ):

$$\sigma \models p \quad \Leftrightarrow \quad p \in \sigma_0$$

$$\sigma \models \varphi \wedge \psi \quad \Leftrightarrow \quad \sigma \models \varphi \text{ and } \sigma \models \psi$$

$$\sigma \models \neg \varphi \quad \Leftrightarrow \quad \sigma \not\models \varphi$$

$$\sigma \models \mathbf{X} \varphi \quad \Leftrightarrow \quad \sigma|_1 \models \varphi$$

$$\sigma \models \varphi \mathbf{U} \psi \quad \Leftrightarrow \quad \text{There exists an } i \text{ such that } \sigma|_i \models \psi \\ \text{and for all } j < i, \sigma|_j \models \varphi$$

CTL*

Computational Tree Logic – specifies branching computation properties

CTL* Syntax:

- State formulas φ and path formulas π

$$\begin{aligned}\varphi &:= p \parallel \varphi \wedge \varphi \parallel \neg \varphi \parallel \mathbf{A}\pi \parallel \mathbf{E}\pi \\ \pi &:= \varphi \parallel \pi \wedge \pi \parallel \neg \pi \parallel \mathbf{X}\pi \parallel \pi \mathbf{U} \pi\end{aligned}$$

CTL* Semantics:

- Computation trees
- State semantics computed at the root:
 - $t \models \mathbf{A}\pi$ if $\forall \pi \in \text{Paths}(t), \rho \models \pi$
 - $t \models \mathbf{E}\pi$ if $\exists \pi \in \text{Paths}(t), \rho \models \pi$
- Path semantics computed with paths (sequences of trees) similar to LTL

CTL

Computational Tree Logic – specifies branching computation properties

CTL Syntax:

- As per CTL*, but modal operators on path formulas are restricted to state formulas:

$$\begin{aligned}\varphi &:= p \parallel \varphi \wedge \varphi \parallel \neg \varphi \parallel \mathbf{A}\pi \parallel \mathbf{E}\pi \\ \pi &:= \mathbf{X}\varphi \parallel \varphi \mathbf{U} \varphi\end{aligned}$$

- Alternative:

$$\varphi := p \parallel \varphi \wedge \varphi \parallel \neg \varphi \parallel \mathbf{AX}\varphi \parallel \mathbf{EX}\varphi \parallel \mathbf{A}(\varphi \mathbf{U} \varphi) \parallel \mathbf{E}(\varphi \mathbf{U} \varphi)$$

CTL Semantics:

- As per CTL*

Comparing Logics

Formula Equivalence

Two formulae are **equivalent** iff they admit the same models.

$$\frac{\forall A. (A \models P) \Leftrightarrow (A \models Q)}{P \equiv Q}$$

Logic Expressiveness

A logic L_1 is **more expressive** than a logic L_2 , written $L_2 \subseteq L_1$, iff:
For all $\varphi_2 \in L_2$, there is a $\varphi_1 \in L_1$ such that $\varphi_1 \equiv \varphi_2$.

CTL $\subseteq$ CTL*? LTL $\subseteq$ CTL*? LTL $\subseteq$ CTL? CTL $\subseteq$ LTL?

$$\text{LTL} \subseteq \text{CTL}^*$$

LTL formulae look like CTL^* *path formulae*. How do we convert them into equivalent *state formulae*?

Recall that $A \models \varphi$ iff $\forall \rho \in \text{Traces}(A). \rho \models \varphi$

$$\text{LTL} \subseteq \text{CTL}^*$$

LTL formulae look like CTL^* *path formulae*. How do we convert them into equivalent *state formulae*?

Recall that $A \models \varphi$ iff $\forall \rho \in \text{Traces}(A). \rho \models \varphi$

For all LTL formulae φ :

$$A \models_{\text{LTL}} \varphi \iff A \models_{\text{CTL}^*} \mathbf{A} \varphi$$

Proof follows trivially from the definition of $\mathbf{A}$.

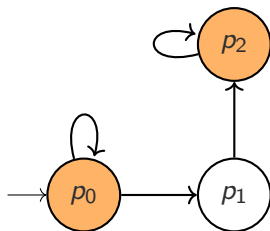
CTL $\subseteq$ LTL?

CTL Formula: **AF AG** ●

CTL $\subseteq$ LTL?

CTL Formula: **AF AG** ●

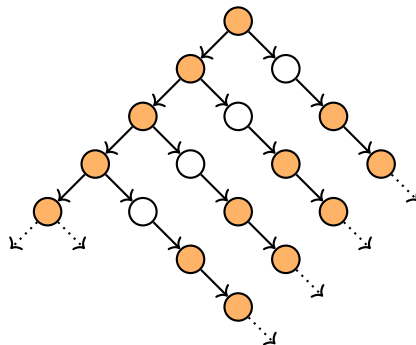
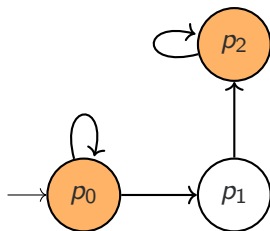
LTL Formula: **FG** ●? *does this work?*



CTL $\subseteq$ LTL?

CTL Formula: **AF AG** ●

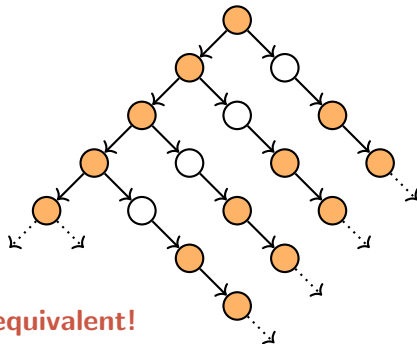
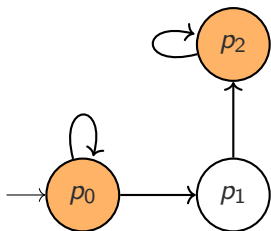
LTL Formula: **FG** ●? *does this work?*



CTL $\subseteq$ LTL?

CTL Formula: **AF AG** ●

LTL Formula: **FG** ●? *does this work?*



It's not equivalent!

CTL $\not\subseteq$ LTL

Let's prove it.

CTL $\not\subseteq$ LTL

Let's prove it.

Lemma (Trace Inclusion)

If $\text{Traces}(A) \subseteq \text{Traces}(B)$ then for any LTL formula φ ,
 $B \models \varphi \implies A \models \varphi$

CTL $\not\subseteq$ LTL

Let's prove it.

Lemma (Trace Inclusion)

If $\text{Traces}(A) \subseteq \text{Traces}(B)$ then for any LTL formula φ ,
 $B \models \varphi \implies A \models \varphi$

Suppose $\exists$ an LTL formula φ that is equivalent to **AG EF ●**.

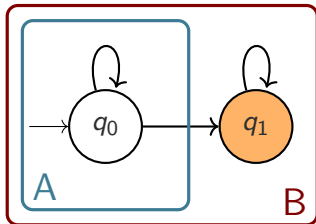
CTL $\not\subseteq$ LTL

Let's prove it.

Lemma (Trace Inclusion)

If $\text{Traces}(A) \subseteq \text{Traces}(B)$ then for any LTL formula φ ,
 $B \models \varphi \implies A \models \varphi$

Suppose $\exists$ an LTL formula φ that is equivalent to **AG EF** ●.



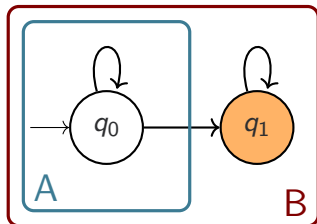
CTL $\not\subseteq$ LTL

Let's prove it.

Lemma (Trace Inclusion)

If $\text{Traces}(A) \subseteq \text{Traces}(B)$ then for any LTL formula φ ,
 $B \models \varphi \implies A \models \varphi$

Suppose $\exists$ an LTL formula φ that is equivalent to **AG EF ●**.



Proof

Observe that $B \models \mathbf{AG\ EF\ \bullet}$ but
 $A \not\models \mathbf{AG\ EF\ \bullet}$

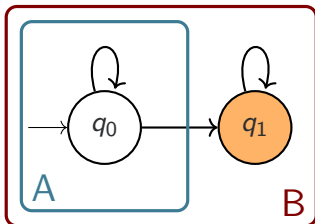
CTL $\not\subseteq$ LTL

Let's prove it.

Lemma (Trace Inclusion)

If $\text{Traces}(A) \subseteq \text{Traces}(B)$ then for any LTL formula φ ,
 $B \models \varphi \implies A \models \varphi$

Suppose $\exists$ an LTL formula φ that is equivalent to **AG EF ●**.



Proof

Observe that $B \models \mathbf{AG\ EF\ \bullet}$ but
 $A \not\models \mathbf{AG\ EF\ \bullet}$

Because φ is equivalent, we know
 $B \models \varphi$ and $A \not\models \varphi$.

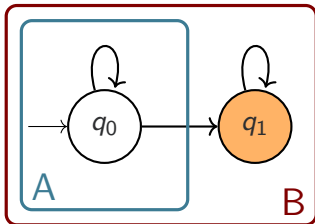
CTL $\not\subseteq$ LTL

Let's prove it.

Lemma (Trace Inclusion)

If $\text{Traces}(A) \subseteq \text{Traces}(B)$ then for any LTL formula φ ,
 $B \models \varphi \implies A \models \varphi$

Suppose $\exists$ an LTL formula φ that is equivalent to **AG EF ●**.



Proof

Observe that $B \models \mathbf{AG\ EF\ } \bullet$ but
 $A \not\models \mathbf{AG\ EF\ } \bullet$

Because φ is equivalent, we know
 $B \models \varphi$ and $A \not\models \varphi$.

But, as $\text{Traces}(A) \subseteq \text{Traces}(B)$, our lemma says that $A \models \varphi$.

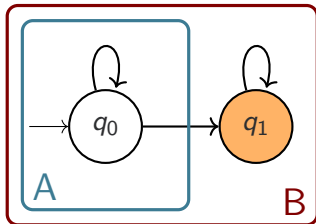
CTL $\not\subseteq$ LTL

Let's prove it.

Lemma (Trace Inclusion)

If $\text{Traces}(A) \subseteq \text{Traces}(B)$ then for any LTL formula φ ,
 $B \models \varphi \implies A \models \varphi$

Suppose $\exists$ an LTL formula φ that is equivalent to **AG EF ●**.



Proof

Observe that $B \models \mathbf{AG\ EF\ \bullet}$ but
 $A \not\models \mathbf{AG\ EF\ \bullet}$

Because φ is equivalent, we know
 $B \models \varphi$ and $A \not\models \varphi$.

But, as $\text{Traces}(A) \subseteq \text{Traces}(B)$, our
lemma says that $A \models \varphi$.

Contradiction!

LTL $\subseteq$ CTL?

LTL Formula: **F (● $\wedge$ X ●)**

$LTL \subseteq CTL?$

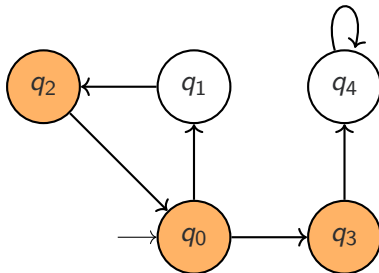
LTL Formula: $F (\bullet \wedge X \bullet)$

CTL Formula: $AF (\bullet \wedge AX \bullet)$. Does this work?

LTL $\subseteq$ CTL?

LTL Formula: **F** (● $\wedge$ **X** ●)

CTL Formula: **AF** (● $\wedge$ **AX** ●). Does this work?



Nope!

LTL $\not\subseteq$ CTL

Lemma

It is possible to construct two families of automata A_i and B_i such that:

- They are distinguished by the LTL formula $\mathbf{F\ G\ } \bullet$, that is:
 $A_i \models \mathbf{F\ G\ } \bullet$ but $B_i \not\models \mathbf{F\ G\ } \bullet$ for any i .

LTL $\not\subseteq$ CTL

Lemma

It is possible to construct two families of automata A_i and B_i such that:

- They are distinguished by the LTL formula $\mathbf{F G} \bullet$, that is:
 $A_i \models \mathbf{F G} \bullet$ but $B_i \not\models \mathbf{F G} \bullet$ for any i .
- They cannot be distinguished by CTL formulae of length $\leq i$.
That is, $\forall i. \forall \varphi. |\varphi| \leq i \Rightarrow (A_i \models \varphi \Leftrightarrow B_i \models \varphi)$

See the textbook (Baier and Katoen) for details.

LTL $\not\subseteq$ CTL

Lemma

It is possible to construct two **families of automata** A_i and B_i such that:

- They are distinguished by the LTL formula **F G ●**, that is:
 $A_i \models \mathbf{F G \bullet}$ but $B_i \not\models \mathbf{F G \bullet}$ for any i .
- They cannot be distinguished by CTL formulae of **length** $\leq i$.
That is, $\forall i. \forall \varphi. |\varphi| \leq i \Rightarrow (A_i \models \varphi \Leftrightarrow B_i \models \varphi)$

See the textbook (Baier and Katoen) for details.

Proof

Let φ be a CTL formula equivalent to **F G ●**.

LTL $\not\subseteq$ CTL

Lemma

It is possible to construct two families of automata A_i and B_i such that:

- They are distinguished by the LTL formula $\mathbf{F G} \bullet$, that is:
 $A_i \models \mathbf{F G} \bullet$ but $B_i \not\models \mathbf{F G} \bullet$ for any i .
- They cannot be distinguished by CTL formulae of length $\leq i$.
That is, $\forall i. \forall \varphi. |\varphi| \leq i \Rightarrow (A_i \models \varphi \Leftrightarrow B_i \models \varphi)$

See the textbook (Baier and Katoen) for details.

Proof

Let φ be a CTL formula equivalent to $\mathbf{F G} \bullet$. Let k be the length of φ , i.e. $k = |\varphi|$.

LTL $\not\subseteq$ CTL

Lemma

It is possible to construct two **families of automata** A_i and B_i such that:

- They are distinguished by the LTL formula **F G ●**, that is:
 $A_i \models \mathbf{F\,G\,}\bullet$ but $B_i \not\models \mathbf{F\,G\,}\bullet$ for any i .
- They cannot be distinguished by CTL formulae of **length** $\leq i$.
That is, $\forall i. \forall \varphi. |\varphi| \leq i \Rightarrow (A_i \models \varphi \Leftrightarrow B_i \models \varphi)$

See the textbook (Baier and Katoen) for details.

Proof

Let φ be a CTL formula equivalent to **F G ●**. Let k be the length of φ , i.e. $k = |\varphi|$. From lemma, $A_k \models \mathbf{F\,G\,}\bullet$ and $B_k \not\models \mathbf{F\,G\,}\bullet$,

LTL $\not\subseteq$ CTL

Lemma

It is possible to construct two **families of automata** A_i and B_i such that:

- They are distinguished by the LTL formula **F G ●**, that is:
 $A_i \models \mathbf{F\,G\,}\bullet$ but $B_i \not\models \mathbf{F\,G\,}\bullet$ for any i .
- They cannot be distinguished by CTL formulae of **length** $\leq i$.
That is, $\forall i. \forall \varphi. |\varphi| \leq i \Rightarrow (A_i \models \varphi \Leftrightarrow B_i \models \varphi)$

See the textbook (Baier and Katoen) for details.

Proof

Let φ be a CTL formula equivalent to **F G ●**. Let k be the length of φ , i.e. $k = |\varphi|$. From lemma, $A_k \models \mathbf{F\,G\,}\bullet$ and $B_k \not\models \mathbf{F\,G\,}\bullet$, but also $A_k \models \varphi \Leftrightarrow B_k \models \varphi$,

LTL $\not\subseteq$ CTL

Lemma

It is possible to construct two families of automata A_i and B_i such that:

- They are distinguished by the LTL formula $\mathbf{F G} \bullet$, that is:
 $A_i \models \mathbf{F G} \bullet$ but $B_i \not\models \mathbf{F G} \bullet$ for any i .
- They cannot be distinguished by CTL formulae of length $\leq i$.
That is, $\forall i. \forall \varphi. |\varphi| \leq i \Rightarrow (A_i \models \varphi \Leftrightarrow B_i \models \varphi)$

See the textbook (Baier and Katoen) for details.

Proof

Let φ be a CTL formula equivalent to $\mathbf{F G} \bullet$. Let k be the length of φ , i.e. $k = |\varphi|$. From lemma, $A_k \models \mathbf{F G} \bullet$ and $B_k \not\models \mathbf{F G} \bullet$, but also $A_k \models \varphi \Leftrightarrow B_k \models \varphi$, so φ cannot be equivalent.

Contradiction!

$$\text{CTL} \subset \text{CTL}^*$$

Every CTL formula is also a CTL* formula. But is it a **strict** inclusion (i.e. $\text{CTL} \subset \text{CTL}^*$)?

$$\text{CTL} \subset \text{CTL}^*$$

Every CTL formula is also a CTL* formula. But is it a **strict** inclusion (i.e. $\text{CTL} \subset \text{CTL}^*$)?

Yes.

$$\text{CTL} \subset \text{CTL}^*$$

Every CTL formula is also a CTL* formula. But is it a **strict** inclusion (i.e. $\text{CTL} \subset \text{CTL}^*$)?

Yes. We know already that $\text{LTL} \subseteq \text{CTL}^*$ and that $\text{LTL} \not\subseteq \text{CTL}$. So pick any LTL formula that cannot be expressed in CTL, and we have a formula that cannot be expressed in CTL but can be in CTL*.

$$\text{LTL} \subset \text{CTL}^*$$

We saw that $\text{LTL} \subseteq \text{CTL}^*$. But is it a **strict** inclusion?
(i.e. $\text{LTL} \subset \text{CTL}^*$)?

$$\text{LTL} \subset \text{CTL}^*$$

We saw that $\text{LTL} \subseteq \text{CTL}^*$. But is it a **strict** inclusion?
(i.e. $\text{LTL} \subset \text{CTL}^*$)?

Yes.

$$\text{LTL} \subset \text{CTL}^*$$

We saw that $\text{LTL} \subseteq \text{CTL}^*$. But is it a **strict** inclusion?
(i.e. $\text{LTL} \subset \text{CTL}^*$)?

Yes. We know already that $\text{CTL} \subseteq \text{CTL}^*$ and that $\text{CTL} \not\subseteq \text{LTL}$.
So pick any CTL formula that cannot be expressed in LTL, and we have a formula that cannot be expressed in LTL but can be in CTL^* .

$$(LTL \cup CTL) \subset CTL^*$$

Is there any formula that **can** be expressed in CTL^* but not in CTL nor in LTL?

$$(\text{LTL} \cup \text{CTL}) \subset \text{CTL}^*$$

Is there any formula that **can** be expressed in CTL^* but not in CTL nor in LTL?

Strict Inclusion

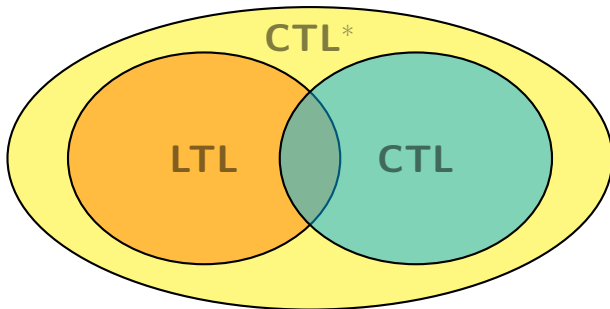
Yes. The proof is very involved, but the formula **E G F ●** cannot be expressed in either LTL nor CTL.

$$(\text{LTL} \cup \text{CTL}) \subset \text{CTL}^*$$

Is there any formula that **can** be expressed in CTL^* but not in CTL nor in LTL?

Strict Inclusion

Yes. The proof is very involved, but the formula **E G F** ● cannot be expressed in either LTL nor CTL.



The CTL Model Checking Problem

Given

- A CTL formula φ , and
- An automaton A ,

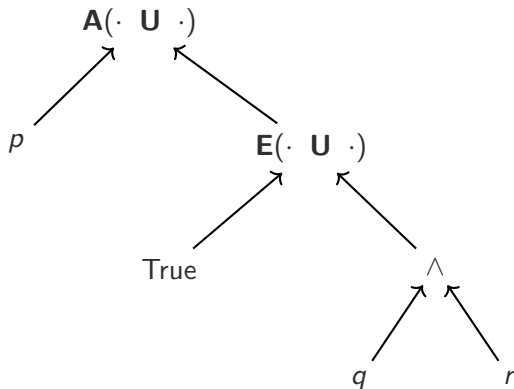
Determine if $A \models \varphi$.

Our approach

We first break the formula up into a *parse tree*. Then, **annotate states** in a bottom-up fashion with the (sub-)formulae they satisfy.

Parse Trees

$$A(p \text{ U } E(\text{True} \text{ U } q \wedge r))$$



Formal Algorithm: Basic Propositions

```
case  $\varphi \in \mathcal{P}$  do                                     /* Atomic proposition */
┌   foreach  $q \in Q$  do
│   if  $\varphi \in L(q)$  then
│   │    $q.\varphi := \text{True};$ 
│   else
│   │    $q.\varphi := \text{False};$ 
└

case  $\varphi = \neg\psi$  do                                   /* Negation */
┌   Mark( $A, \psi$ ) ;
│   foreach  $q \in Q$  do
│   │    $q.\varphi := \neg q.\psi$  ;
└

case  $\varphi = \psi_1 \wedge \psi_2$  do                         /* Conjunction */
┌   Mark( $A, \psi_1$ ); Mark( $A, \psi_2$ ) ;
│   foreach  $q \in Q$  do
│   │    $q.\varphi := q.\psi_1 \wedge q.\psi_2$  ;
└
```

Formal Algorithm: EX

```

case  $\varphi = \mathbf{EX} \psi$  do                                /* Exists a Successor */
┌   Mark( $A, \psi$ ) ;
┌   foreach  $q \in Q$  do
┌        $q.\varphi := \text{False}$ ;
┌   foreach  $(q, q') \in \delta$  do
┌       if  $q'.\psi$  then
┌            $q.\varphi := \text{True}$  ;
└
└
└

```

We can simplify $\mathbf{AX} \psi$ to $\neg \mathbf{EX} \neg \psi$. Why?

```

case  $\varphi = \mathbf{E} \psi_1 \mathbf{U} \psi_2$  do                                     /* Exist Until */
|
|   Mark( $A, \psi_1$ ) ; Mark( $A, \psi_2$ ) ;
|   foreach  $q \in Q$  do
|   |
|   |    $q.\varphi := \text{False};$ 
|   |    $q.\text{visited} := \text{False};$ 
|   |   if  $q.\psi_2$  then
|   |   |
|   |   |    $q.\varphi := \text{True};$ 
|   |   |    $q.\text{visited} := \text{True};$ 
|   |   |    $W := W \cup \{q\};$ 
|   |
|
|   while  $W \neq \emptyset$  do
|   |
|   |    $q := \text{pop}(W);$  /*  $q$  satisfies  $\varphi$  */
|   |   foreach  $(q', q) \in \delta$  do
|   |   |
|   |   |   if  $\neg q'.\text{visited}$  then
|   |   |   |
|   |   |   |    $q'.\text{visited} := \text{True};$ 
|   |   |   |   if  $q'.\psi_1$  then
|   |   |   |   |
|   |   |   |   |    $q'.\varphi := \text{True}; W := W \cup \{q'\};$ 
|   |   |
|   |
|
|

```

```
case  $\varphi = \mathbf{A} \psi_1 \mathbf{U} \psi_2$  do                               /* For All Until */
  Mark( $A, \psi_1$ ) ; Mark( $A, \psi_2$ );
  foreach  $q \in Q$  do
     $q.\varphi := \text{False}$ ;
     $q.nbUnchecked := |\delta(q)|$ ;
    if  $q.\psi_2$  then
       $q.\varphi := \text{True}$  ;
       $W := W \cup \{q\}$ ;
  while  $W \neq \emptyset$  do
     $q := \text{pop}(W)$ ;
    /*  $q$  satisfies  $\varphi$  */
    foreach  $(q', q) \in \delta$  do
       $q'.nbUnchecked := q'.nbUnchecked - 1$  ;
      if  $(q'.nbUnchecked = 0 \wedge q'.\psi_1 \wedge \neg q'.\varphi)$  then
         $q'.\varphi := \text{True}$  ;
         $W := W \cup \{q'\}$ ;
```

Complexity?

Assume a fixed size of formula $|\varphi|$, what is the run time complexity of this algorithm?

Complexity?

Assume a fixed size of formula $|\varphi|$, what is the run time complexity of this algorithm?

- Complexity for atomic propositions, $\wedge$ and $\neg$:

Complexity?

Assume a fixed size of formula $|\varphi|$, what is the run time complexity of this algorithm?

- Complexity for atomic propositions, $\wedge$ and $\neg$: $\mathcal{O}(|Q|)$
- Complexity for **EX**:

Complexity?

Assume a fixed size of formula $|\varphi|$, what is the run time complexity of this algorithm?

- Complexity for atomic propositions, $\wedge$ and $\neg$: $\mathcal{O}(|Q|)$
- Complexity for **EX**: $\mathcal{O}(|Q|)$
- Complexity for **E**($\cdot$ **U** $\cdot$):

Complexity?

Assume a fixed size of formula $|\varphi|$, what is the run time complexity of this algorithm?

- Complexity for atomic propositions, $\wedge$ and $\neg$: $\mathcal{O}(|Q|)$
- Complexity for **EX**: $\mathcal{O}(|Q|)$
- Complexity for **E**($\cdot$ **U** $\cdot$): $\mathcal{O}(|Q| + |\delta|)$
- Complexity for **A**($\cdot$ **U** $\cdot$):

Complexity?

Assume a fixed size of formula $|\varphi|$, what is the run time complexity of this algorithm?

- Complexity for atomic propositions, $\wedge$ and $\neg$: $\mathcal{O}(|Q|)$
- Complexity for **EX**: $\mathcal{O}(|Q|)$
- Complexity for **E**($\cdot$ **U** $\cdot$): $\mathcal{O}(|Q| + |\delta|)$
- Complexity for **A**($\cdot$ **U** $\cdot$): $\mathcal{O}(|Q| + |\delta|)$

Complexity?

Assume a fixed size of formula $|\varphi|$, what is the run time complexity of this algorithm?

- Complexity for atomic propositions, $\wedge$ and $\neg$: $\mathcal{O}(|Q|)$
- Complexity for **EX**: $\mathcal{O}(|Q|)$
- Complexity for **E**($\cdot$ **U** $\cdot$): $\mathcal{O}(|Q| + |\delta|)$
- Complexity for **A**($\cdot$ **U** $\cdot$): $\mathcal{O}(|Q| + |\delta|)$

Therefore, overall complexity is:

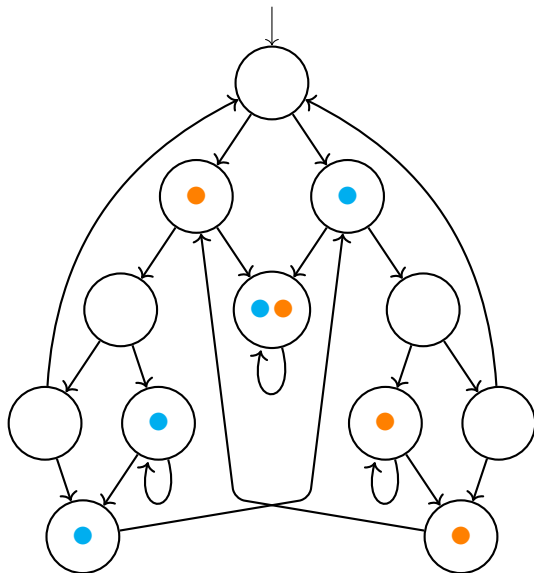
Complexity?

Assume a fixed size of formula $|\varphi|$, what is the run time complexity of this algorithm?

- Complexity for atomic propositions, $\wedge$ and $\neg$: $\mathcal{O}(|Q|)$
- Complexity for **EX**: $\mathcal{O}(|Q|)$
- Complexity for **E**($\cdot$ **U** $\cdot$): $\mathcal{O}(|Q| + |\delta|)$
- Complexity for **A**($\cdot$ **U** $\cdot$): $\mathcal{O}(|Q| + |\delta|)$

Therefore, overall complexity is: $\mathcal{O}((|Q| + |\delta|) \times |\varphi|)$

Example



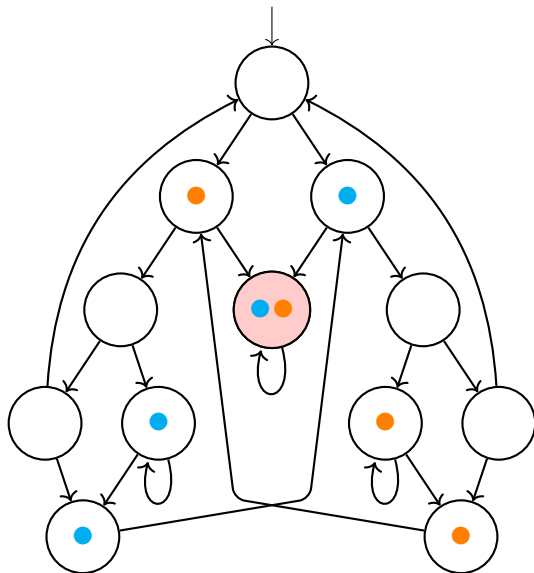
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- $EF (\text{blue} \wedge \text{orange})$

Example



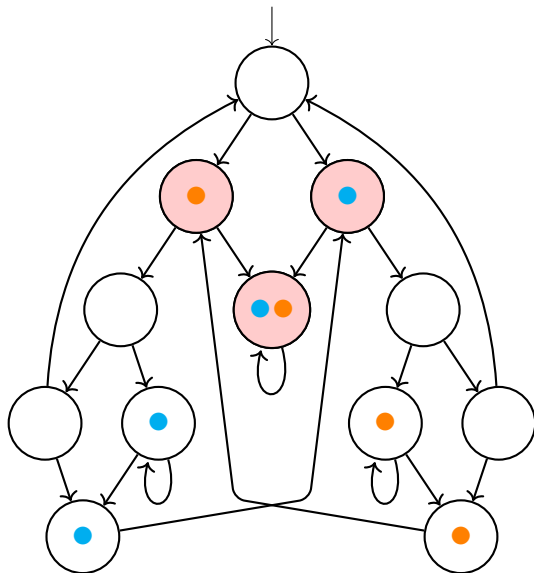
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- $EF (\text{blue} \wedge \text{orange})$

Example



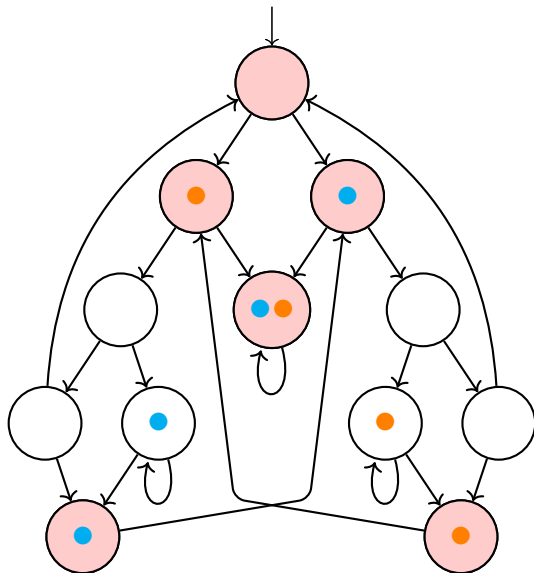
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- $EF (\text{blue} \wedge \text{orange})$

Example



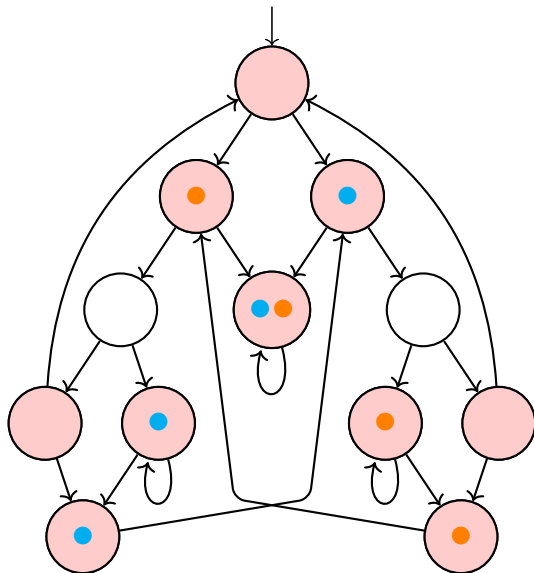
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- $EF (\text{blue} \wedge \text{orange})$

Example



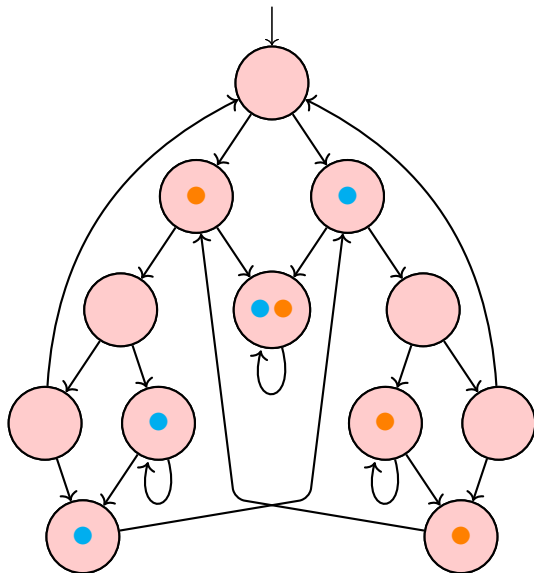
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- $EF (\text{blue} \wedge \text{orange})$

Example



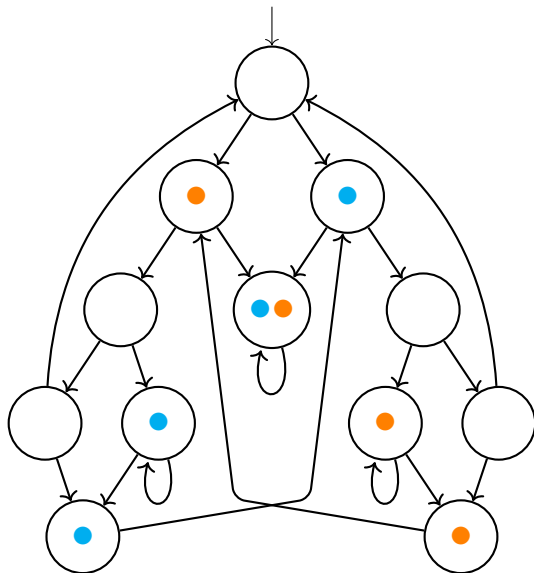
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- $EF (\text{blue dot} \wedge \text{orange dot})$

Example



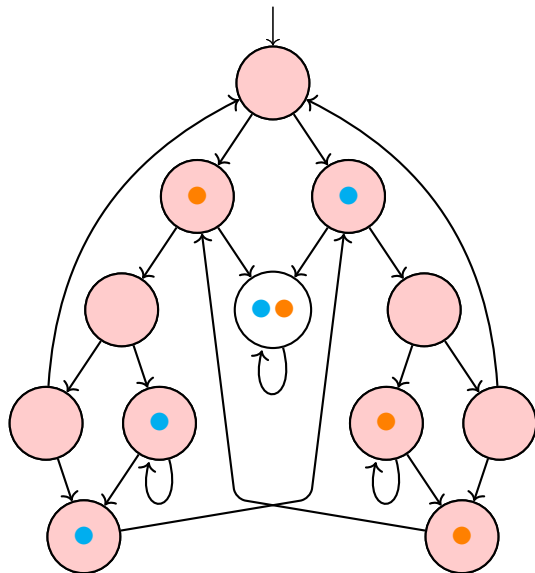
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- **EF** (● $\wedge$ ●)
- **EF AG** (● $\wedge$ ●)

Example



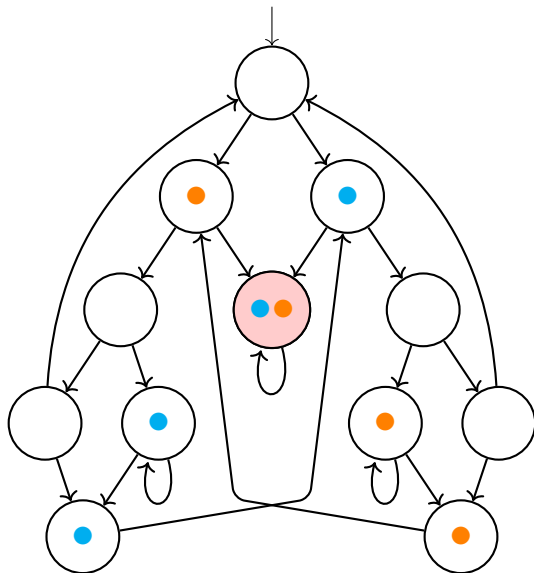
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- $EF (\text{blue} \wedge \text{orange})$
- $EF AG (\text{blue} \wedge \text{orange})$

Example



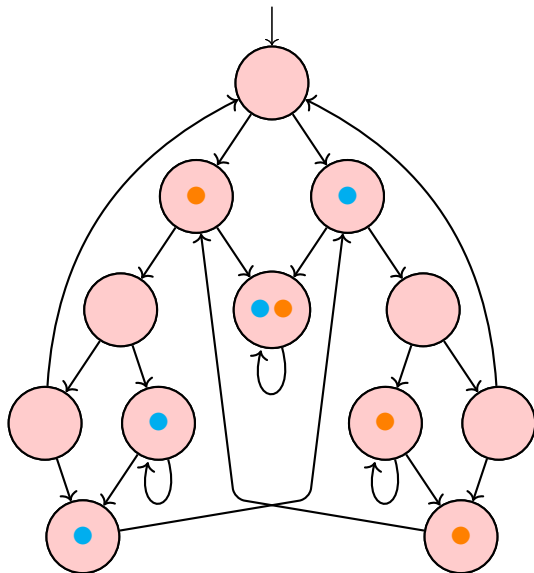
Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- **EF** (● $\wedge$ ●)
- **EF AG** (● $\wedge$ ●)

Example



Procedure

- **Simplify** to basic CTL operations.
- **Build** parse tree for new formula.
- **Mark** states bottom up as described.

Example

- $EF (\text{blue} \wedge \text{orange})$
- $EF AG (\text{blue} \wedge \text{orange})$

Bibliography

Expressiveness:

- Huth/Ryan: Logic in Computer Science, Section 3.5
- Baier/Katoen: Principles of Model Checking, Section 6.3

CTL Model Checking

- Bérard et al: System and Software Verification, Section 3.1
- Baier/Katoen: Principles of Model Checking, Section 6.4
- Clarke et al: Model Checking, Section 4.1
- Huth/Ryan: Logic in Computer Science, Section 3.6